



ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**Student Hostel Management System — Automated Room Allocation & Fee Tracking**” is submitted by

Nunna Sai Sri Laxmi (25B11CS682)

Kamuju Sree Reshma Laxmi (25B11CS385)

Bheemireddy Aswini (25B11CS107)

**Medepalli Akshaya
Keerthana (25B11CS592)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

Project Guide

Mrs. K Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the Department

Dr. Phani Sridhar Addepalli, Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled “**Student Hostel Management System — Automated Room Allocation & Fee Tracking**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree in Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date: 29-09-2026

By

Nunna Sai Sri Laxmi (25B11CS682)
Kamuju Sree Reshma Laxmi (25B11CS385)
Bheemireddy Aswini (25B11CS107)
Medepalli Akshaya Keerthana (25B11CS592)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. K Vydehi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor, Dr Hari Jyothula, Associate Professor and Dr Pennada Siva Satya Prasad, Assistant Profeesor**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Addepalli, Assistant Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

1. Abstract	8
2. Introduction	9
3. System Analysis	10
4. Database Design	11
5. ER Diagram	12
6. Relational Schema	13
7. Table Design	14–15
8. Normalization	16–20
9. SQL Implementation	21–24
10. SQL Queries and Outputs	25–27
11. Conclusion	28
12. Future Enhancements	28
13. References	29

LIST OF FIGURES

Figure 1. ER representation of the Student Hostel Management System 12



LIST OF TABLES

Table	Page No.
Table 1. Major entities and their purposes	12
Table 2. Relationships and cardinalities	12
Table 3. Relational schema	14
Table 4. WARDEN table structure	15
Table 5. ROOM table structure	15
Table 6. STUDENT table structure	15
Table 7. FEE_PAYMENT table structure	15–16
Table 8. COMPLAINT table structure	16
Table 9. VISITOR table structure	16
Table 10. Sample SQL query outputs	25–27

ACRONYMS

DBMS – Database Management System

ER – Entity Relationship

SQL – Structured Query Language

DDL – Data Definition Language

DML – Data Manipulation Language

DQL – Data Query Language

DCL – Data Control Language

TCL – Transaction Control Language

PK – Primary Key

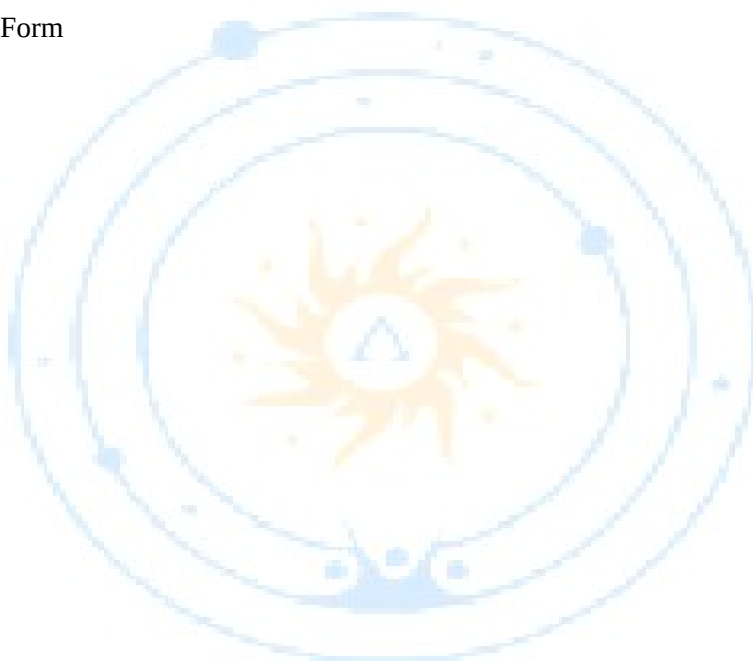
FK – Foreign Key

UNIQUE – Unique Constraint

1NF – First Normal Form

2NF – Second Normal Form

3NF – Third Normal Form



1. ABSTRACT

The Student Hostel Management System is a centralized relational database designed to manage hostel operations such as student registration, room allocation, fee payment tracking, complaint management, and visitor records. The supplied project identifies common problems in manual hostel administration, including room allocation errors, delayed fee tracking, untracked complaints, and missing visitor records. The proposed database uses primary keys, foreign keys, UNIQUE and CHECK constraints, and a normalized relational design. The supplied implementation decomposes the design into 13 tables: six core entity tables, five multi-valued attribute tables, and two relationship tables. The design is normalized up to Third Normal Form (3NF), reducing redundancy and insertion, update, and deletion anomalies. The implementation is written for Oracle SQL and includes table creation, foreign-key relationships, sample data, and 15 SQL queries covering selection, joins, filtering, aggregation, grouping, HAVING, and reporting.



2. INTRODUCTION

2.1 Project Overview

The Student Hostel Management System provides a structured database for hostel administration. It maintains information about wardens, rooms, students, fee payments, complaints, and visitors. The central database allows hostel staff to retrieve information consistently instead of depending on separate paper records.

2.2 Problem Statement

- Manual room assignment can cause overbooking or incorrectly marked free beds.
- Fee dues recorded on paper can be delayed or missed.
- Complaints may be logged informally without a defined status or resolution trail.
- Visitor entries may be left unrecorded, creating an administrative and security gap.
- Repeated information in manual records increases inconsistency and maintenance effort.

2.3 Objectives

- Maintain a single authoritative student record.
- Support room allotment using room capacity and availability.
- Track fee payments, due dates, and payment status.
- Maintain a structured complaint workflow.
- Maintain a visitor log linked to the student being visited.
- Normalize the database to 3NF and enforce referential integrity.
- Support useful SQL reports using joins and aggregate functions.

2.4 Scope

The supplied project scope covers student, room, warden, fee payment, complaint, and visitor management. The database can be extended later with additional hostel blocks, automated user interfaces, stronger access control, audit logging, backups, analytics dashboards, and other administrative modules.

3. SYSTEM ANALYSIS

3.1 Existing System

- Manual record maintenance.
- Separate or disconnected records.
- Room allocation errors caused by lack of a central occupancy view.
- Delayed fee reconciliation.
- Informal complaint tracking.
- Visitor records may be incomplete.

3.2 Proposed System

The proposed system uses a centralized Oracle relational database. Related information is separated into tables and connected through primary and foreign keys. Multi-valued attributes such as student phone numbers and emails are stored in separate child tables. Complaint resolutions and visitor purposes are also separated into relationship tables.

3.3 Functional Requirements

Module	Functional Requirement
Student Management	Register, update, search and display student details.
Room Management	Store room capacity, type, floor, status and warden assignment.
Fee Management	Record amount, payment date, due date and payment status.
Complaint Management	File complaints, assign them to wardens and store resolutions.
Visitor Management	Record visitors, dates, student being visited, phone numbers and purposes.
Reporting	Use SQL joins, grouping, filtering and aggregates to generate hostel reports.

3.4 Non-Functional Requirements

- Data integrity through PK/FK, UNIQUE and CHECK constraints.
- Security through controlled database access for administrative users.
- Reliability during admissions, fee collection and routine hostel operations.
- Performance for room availability and fee-status lookups.
- Usability for hostel staff who may not be database specialists.
- Scalability as the number of students, rooms and records grows.

3.5 End Users

User	Main Interaction
Hostel Warden / Admin	Register students, allot rooms, verify fees, manage complaints and generate reports.
Students	View room, fee and complaint information and file complaints.
Accounts / Fee Office	Record fee payments and track dues.
Security / Gate Staff	Log visitor entries and exits.
System Administrator	Manage user accounts, backups and configuration.



4. DATABASE DESIGN

4.1 Major Entities

Entity	Purpose
Warden	Stores warden identity information.
Room	Stores room number, floor, type, capacity, status and assigned warden.
Student	Stores student identity, course, year and room allocation.
Fee_Payment	Stores hostel fee amount, dates and payment status.
Complaint	Stores complaints raised by students and assigned wardens.
Visitor	Stores visitor details and the student being visited.
Student_Phone / Student_Email	Stores multi-valued student contact information.
Warden_Phone / Warden_Email	Stores multi-valued warden contact information.
Visitor_Phone	Stores multi-valued visitor phone information.
Complaint_Resolution	Stores complaint resolution events.
Visit_Purpose	Stores visitor purpose values.

4.2 Relationships and Cardinality

Relationship	Cardinality	Description
Warden — Room	1:M	One warden can manage multiple rooms; each room has one assigned warden in the supplied design.
Room — Student	1:M	A room can house multiple students up to its capacity; a student is linked to a room.
Student — Fee_Payment	1:M	A student can have multiple payment records.
Student — Complaint	1:M	A student can raise multiple complaints.
Warden — Complaint	1:M	A warden can handle multiple complaints.
Student — Visitor	1:1 in supplied implementation	The SQL includes UNIQUE(student_id) on Visitor, so the implementation restricts a student to one visitor record.

Note: The 1:1 Student–Visitor relationship above reflects the supplied SQL constraint UNIQUE(student_id). The PPT describes visitor logging operationally; the implemented constraint should be retained or changed only if the project team decides to support multiple visitor records per student.

5. ER DIAGRAM

The following diagram is a documentation representation derived from the entities, foreign keys, and relationships in the supplied PPT and Oracle SQL code.

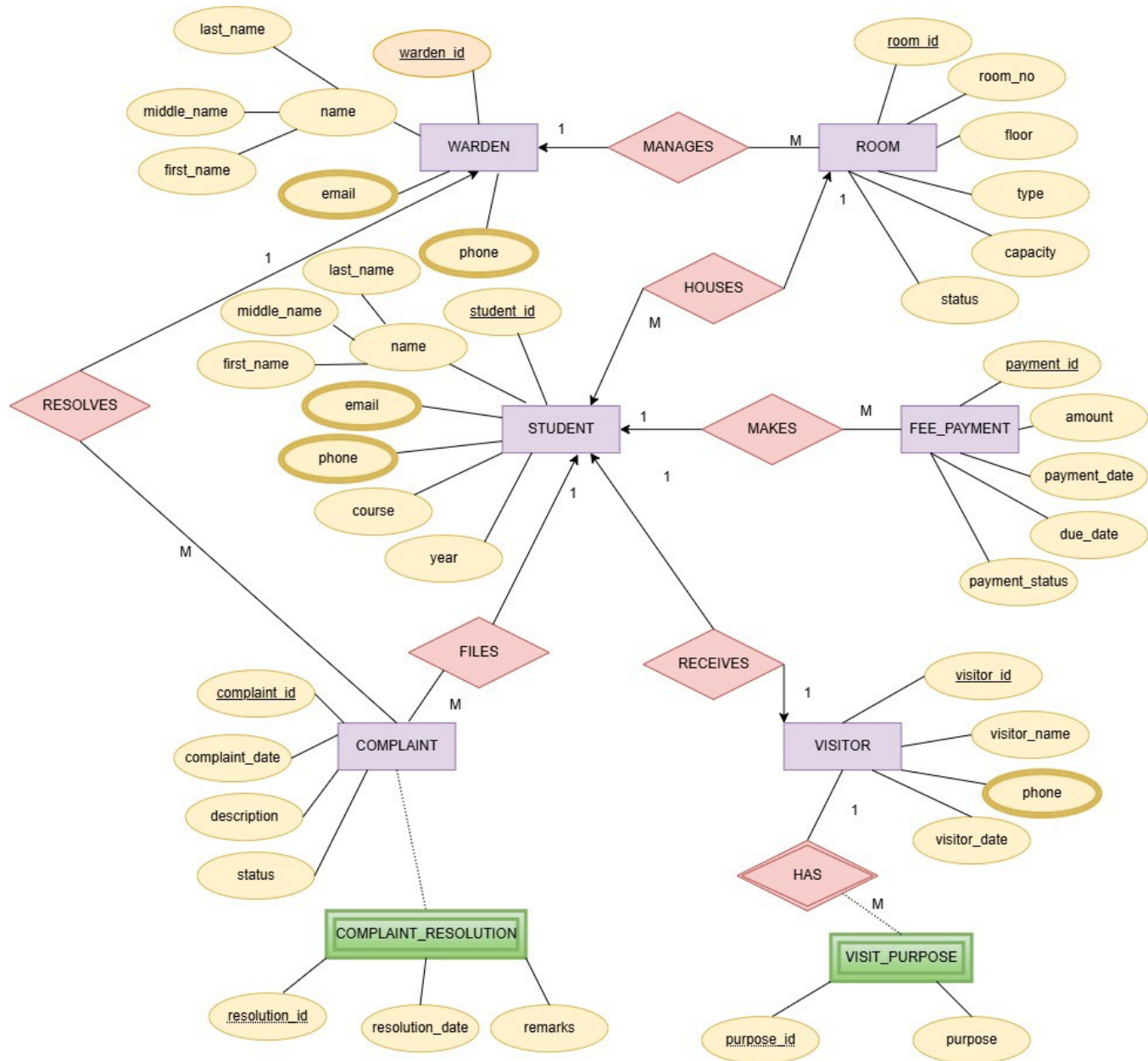


Figure 1: ER representation of the Student Hostel Management System

6. RELATIONAL SCHEMA

Relation	Relational Schema
WARDEN	warden_id (PK), first_name, middle_name, last_name
WARDEN_EMAIL	warden_id (PK/FK), email (PK)
WARDEN_PHONE	warden_id (PK/FK), phone (PK)
ROOM	room_id (PK), room_no, floor, type, capacity, status, warden_id (FK)
STUDENT	student_id (PK), first_name, middle_name, last_name, course, year, room_id (FK)
STUDENT_EMAIL	student_id (PK/FK), email (PK)
STUDENT_PHONE	student_id (PK/FK), phone (PK)
FEE_PAYMENT	payment_id (PK), amount, payment_date, due_date, payment_status, student_id (FK)
COMPLAINT	complaint_id (PK), complaint_date, description, status, student_id (FK), warden_id (FK)
COMPLAINT_RESOLUTION	complaint_id (PK/FK), resolution_id (PK), resolution_date, remarks
VISITOR	visitor_id (PK), visitor_name, visitor_date, student_id (FK)
VISITOR_PHONE	visitor_id (PK/FK), phone (PK)
VISIT_PURPOSE	visitor_id (PK/FK), purpose_id (PK), purpose

7. TABLE DESIGN

7.1 WARDEN

Attribute	Data Type	Constraint	Description
warden_id	INTEGER	PK	Unique warden identifier
first_name	VARCHAR2(30)	—	First name
middle_name	VARCHAR2(30)	—	Middle name
last_name	VARCHAR2(30)	—	Last name

7.2 ROOM

Attribute	Data Type	Constraint	Description
room_id	INTEGER	PK	Unique room identifier
room_no	INTEGER	—	Room number
floor	INTEGER	—	Floor number
type	VARCHAR2(20)	—	Single, Double or Triple
capacity	INTEGER	—	Maximum occupancy
status	NUMBER(1)	—	Room status flag
warden_id	INTEGER	FK	References Warden

7.3 STUDENT

Attribute	Data Type	Constraint	Description
student_id	INTEGER	PK	Unique student identifier
first_name	VARCHAR2(30)	—	First name
middle_name	VARCHAR2(30)	—	Middle name
last_name	VARCHAR2(30)	—	Last name
course	VARCHAR2(30)	—	Academic course/branch
year	INTEGER	—	Year of study
room_id	INTEGER	FK	References Room

7.4 FEE_PAYMENT

Attribute	Data Type	Constraint	Description
payment_id	INTEGER	PK	Unique payment identifier
amount	INTEGER	—	Fee amount

payment_date	DATE	—	Date paid
due_date	DATE	—	Payment due date
payment_status	NUMBER(1)	—	1 = paid, 0 = pending in supplied data
student_id	INTEGER	FK	References Student

7.5 COMPLAINT

Attribute	Data Type	Constraint	Description
complaint_id	INTEGER	PK	Unique complaint identifier
complaint_date	DATE	—	Date filed
description	VARCHAR2(100)	—	Complaint description
status	NUMBER(1)	—	1 = resolved, 0 = open in supplied queries
student_id	INTEGER	FK	Student raising complaint
warden_id	INTEGER	FK	Warden handling complaint

7.6 VISITOR

Attribute	Data Type	Constraint	Description
visitor_id	INTEGER	PK	Unique visitor identifier
visitor_name	VARCHAR2(30)	—	Visitor name
visitor_date	DATE	—	Visit date
student_id	INTEGER	FK + UNIQUE	Student being visited

7.7 Child / Relationship Tables

Table	Purpose	Primary Key
STUDENT_EMAIL	Stores student email values	(student_id, email)
STUDENT_PHONE	Stores student phone values	(student_id, phone)
WARDEN_EMAIL	Stores warden email values	(warden_id, email)
WARDEN_PHONE	Stores warden phone values	(warden_id, phone)
VISITOR_PHONE	Stores visitor phone values	(visitor_id, phone)
COMPLAINT_RESOLUTION	Stores complaint resolution history	(complaint_id, resolution_id)
VISIT_PURPOSE	Stores visitor purpose values	(visitor_id, purpose_id)



8. NORMALIZATION

WHAT GOES WRONG

DELETION ANOMALY

Delete the last student → room information can disappear.

UPDATE ANOMALY

Change one warden phone → many rows must be updated.

INSERTION ANOMALY

A new room cannot be stored until a student exists.

REDUNDANCY

Ravi Sharma and room facts are repeated across student rows.

BEFORE — UNNORMALIZED (UNF)

```
HoSTEL_RECORD(  
    student_id, student_name, phone1, phone2,  
    room_no, room_type, capacity, warden_id,  
    warden_name, warden_phone, amount, due_date,  
    payment_status  
)
```

Start with one flat table: repeated data creates redundancy and anomalies.

AFTER — 1NF TABLES

1NF — Atomic Values

Each value is atomic; phones can grow without schema changes.

```
Student(student_id, first_name, middle_name,  
last_name, course, year, room_id)  
Student_Email(student_id FK, email)  
Student_Phone(student_id FK, phone)
```

BEFORE — SCHEMA + SAMPLE

```
Student(student_id, name, phone1, phone2, email)
```

Problem: phone1/phone2 is a repeating group.

A third phone would require changing the table structure.

Remove repeating groups and make every attribute single-valued.

AFTER — 2NF TABLES

1NF — Atomic Values

Student details are stored once; payment facts stay in Fee_Payment.

```
Student(student_id, first_name, ..., course)  
Fee_Payment(payment_id, amount, payment_date,  
due_date, payment_status, student_id FK)
```

BEFORE — PARTIAL DEPENDENCY

```
PK = (student_id, payment_id)  
Fee_Detail(student_id, payment_id,  
student_name, course, amount, payment_date)
```

student_name and course depend only on student_id → **partial dependency**.

1NF + every non-key attribute depends on the whole primary key.

AFTER — 3NF TABLES

2NF — No Partial Dependency

BEFORE — SCHEMA

Room(room_id, room_no, floor, type,
capacity, status, warden_id,
warden_name, warden_phone)

This is a **transitive dependency**:

room_id → warden_id → warden_name / warden_phone

AFTER — 3NF TABLES

Warden(warden_id, first_name, middle_name, last_name)

Room(room_id, room_no, floor, type,
capacity, status, warden_id FK)

2NF + non-key attributes depend only on the primary key.

3NF — No Transitive Dependency

BEFORE — FLAT SCHEMA

HOSTEL_RECORD

Student + Phone + Room + Warden + Fee

all stored together

Problems:

- repeated names
- repeated room facts
- anomalies

AFTER — 3NF SCHEMA

Warden(warden_id, first_name, middle_name, last_name)

Room(room_id, room_no, floor, type,
capacity, status, warden_id FK)

Student(student_id, first_name, middle_name,
last_name, course, year, room_id FK)

Fee_Payment(payment_id, amount, payment_date,
due_date, payment_status, student_id FK)

Complaint(complaint_id, complaint_date,
description, status, student_id FK, warden_id FK)

Visitor(visitor_id, visitor_name,
visitor_date, student_id FK)

Student_Phone / Student_Email

Warden_Phone / Warden_Email

Visitor_Phone

Complaint_Resolution

Visit_Purpose

13 tables • PK/FK relationships enforce referential integrity

Result: redundancy ↓ • anomalies removed

keys.

9. SQL IMPLEMENTATION

9.1 DDL — Table Creation

```
CREATE TABLE room
(
    room_id INTEGER,
    room_no INTEGER,
    floor INTEGER,
    type VARCHAR2(20),
    capacity INTEGER,
    status NUMBER(1),
    CONSTRAINT room_pk PRIMARY KEY(room_id)
);

CREATE TABLE fee_payment
(
    payment_id INTEGER,
    amount INTEGER,
    payment_date DATE,
    due_date DATE,
    payment_status NUMBER(1),
    CONSTRAINT fee_payment_pk PRIMARY KEY(payment_id)
);

CREATE TABLE complaint
(
    complaint_id INTEGER,
    complaint_date DATE,
    description VARCHAR2(100),
    status NUMBER(1),
    CONSTRAINT complaint_pk PRIMARY KEY(complaint_id)
);

CREATE TABLE visitor
(
    visitor_id INTEGER,
    visitor_name VARCHAR2(30),
    visitor_date DATE,
    CONSTRAINT visitor_pk PRIMARY KEY(visitor_id)
);

CREATE TABLE warden
(
    warden_id INTEGER,
    first_name VARCHAR2(30),
    middle_name VARCHAR2(30),
    last_name VARCHAR2(30),
    CONSTRAINT warden_pk PRIMARY KEY(warden_id)
);

CREATE TABLE student
(
    student_id INTEGER,
    first_name VARCHAR2(30),
    middle_name VARCHAR2(30),
    last_name VARCHAR2(30),
    course VARCHAR2(30),
    year INTEGER,
    CONSTRAINT student_pk PRIMARY KEY(student_id)
);
```

The supplied implementation then creates the child tables for multi-valued attributes and adds foreign keys to connect the core entities.

```
CREATE TABLE student_phone
(
    student_id INTEGER,
    phone VARCHAR2(15),
    CONSTRAINT student_phone_pk PRIMARY KEY(student_id, phone),
```

```

        CONSTRAINT student_phone_fk FOREIGN KEY(student_id)
            REFERENCES student(student_id)
    );

CREATE TABLE student_email
(
    student_id INTEGER,
    email VARCHAR2(50),
    CONSTRAINT student_email_pk PRIMARY KEY(student_id, email),
    CONSTRAINT student_email_fk FOREIGN KEY(student_id)
        REFERENCES student(student_id)
);

CREATE TABLE warden_phone
(
    warden_id INTEGER,
    phone VARCHAR2(15),
    CONSTRAINT warden_phone_pk PRIMARY KEY(warden_id, phone),
    CONSTRAINT warden_phone_fk FOREIGN KEY(warden_id)
        REFERENCES warden(warden_id)
);

CREATE TABLE warden_email
(
    warden_id INTEGER,
    email VARCHAR2(50),
    CONSTRAINT warden_email_pk PRIMARY KEY(warden_id, email),
    CONSTRAINT warden_email_fk FOREIGN KEY(warden_id)
        REFERENCES warden(warden_id)
);

CREATE TABLE visitor_phone
(
    visitor_id INTEGER,
    phone VARCHAR2(15),
    CONSTRAINT visitor_phone_pk PRIMARY KEY(visitor_id, phone),
    CONSTRAINT visitor_phone_fk FOREIGN KEY(visitor_id)
        REFERENCES visitor(visitor_id)
);

```

9.2 Foreign-Key Alterations

```

ALTER TABLE room ADD warden_id INTEGER;

ALTER TABLE room ADD CONSTRAINT room_warden_fk
FOREIGN KEY (warden_id)
REFERENCES warden(warden_id);

ALTER TABLE student ADD room_id INTEGER;

ALTER TABLE student ADD CONSTRAINT student_room_fk
FOREIGN KEY (room_id)
REFERENCES room(room_id);

ALTER TABLE fee_payment ADD student_id INTEGER;

ALTER TABLE fee_payment ADD CONSTRAINT payment_student_fk
FOREIGN KEY (student_id)
REFERENCES student(student_id);

ALTER TABLE complaint ADD student_id INTEGER;

ALTER TABLE complaint ADD CONSTRAINT complaint_student_fk
FOREIGN KEY (student_id)
REFERENCES student(student_id);

ALTER TABLE complaint ADD warden_id INTEGER;

ALTER TABLE complaint ADD CONSTRAINT complaint_warden_fk
FOREIGN KEY (warden_id)
REFERENCES warden(warden_id);

```

```

ALTER TABLE visitor ADD student_id INTEGER;

ALTER TABLE visitor ADD CONSTRAINT visitor_student_fk
FOREIGN KEY (student_id)
REFERENCES student(student_id);

ALTER TABLE visitor ADD CONSTRAINT visitor_student_uk
UNIQUE(student_id);

```

9.3 Relationship Tables

```

CREATE TABLE complaint_resolution
(
    complaint_id INTEGER,
    resolution_id INTEGER,
    resolution_date DATE,
    remarks VARCHAR2(100),
    PRIMARY KEY (complaint_id, resolution_id),
    FOREIGN KEY (complaint_id)
        REFERENCES complaint(complaint_id)
);

CREATE TABLE visit_purpose
(
    visitor_id INTEGER,
    purpose_id INTEGER,
    purpose VARCHAR2(50),
    PRIMARY KEY (visitor_id, purpose_id),
    FOREIGN KEY (visitor_id)
        REFERENCES visitor(visitor_id)
);

```

9.4 Sample Data

The supplied code loads 15 rows into each core table and associated child/relationship tables. The PPT reports more than 200 records across the 13 tables.

```

INSERT INTO warden VALUES
(1, 'Ravi', 'Kumar', 'Sharma');

INSERT INTO room VALUES
(101, 1, 1, 'Single', 1, 1, 1);

INSERT INTO student VALUES
(201, 'Sai', 'Sri', 'Laxmi', 'CSE', 2, 101);

INSERT INTO fee_payment VALUES
(301, 25000, DATE '2026-01-05',
DATE '2026-01-10', 1, 201);

INSERT INTO student_phone VALUES
(201, '9100000001');

```

9.5 Sample Record Counts

Table	Records in supplied project
Warden	15
Room	15
Student	15
Fee_Payment	15
Complaint	15
Visitor	15

Student_Email / Student_Phone	15 each
Warden_Email / Warden_Phone	15 each
Visitor_Phone	15
Complaint_Resolution	15
Visit_Purpose	15



10. SQL QUERIES AND OUTPUTS

The supplied SQL contains 15 application-oriented queries. They demonstrate single-table selection, joins, filtering, LEFT JOIN, GROUP BY, HAVING, COUNT, SUM, ordering, and complaint/visitor reporting.

Q1. Display all students

Purpose / concept: Returns the complete student register.

```
SELECT *
FROM student;
```

Q2. Student with room

Purpose / concept: Uses an INNER JOIN between Student and Room.

```
SELECT s.student_id,
       s.first_name || ' ' || s.last_name AS student_name,
       s.course, s.year,
       r.room_no, r.floor, r.type
FROM student s
JOIN room r
ON s.room_id = r.room_id;
```

Q3. Students belonging to CSE

Purpose / concept: Filters students using WHERE.

```
SELECT student_id, first_name, last_name,
       course, year
FROM student
WHERE course = 'CSE';
```

Q4. Rooms managed by each warden

Purpose / concept: Joins Warden and Room and sorts by warden.

```
SELECT w.warden_id,
       w.first_name || ' ' || w.last_name AS warden_name,
       r.room_no, r.floor, r.type
FROM warden w
JOIN room r
ON w.warden_id = r.warden_id
ORDER BY w.warden_id;
```

Q5. Students who have paid

Purpose / concept: Shows paid fee records; status 1 is paid in the supplied data.

```
SELECT s.student_id,
       s.first_name || ' ' || s.last_name AS student_name,
       f.amount, f.payment_date, f.payment_status
FROM student s
JOIN fee_payment f
ON s.student_id = f.student_id
WHERE f.payment_status = 1;
```

Q6. Pending fee payments

Purpose / concept: Shows students whose fee status is pending.

```
SELECT s.student_id,
       s.first_name || ' ' || s.last_name AS student_name,
       f.amount, f.due_date
FROM student s
JOIN fee_payment f
ON s.student_id = f.student_id
WHERE f.payment_status = 0;
```

Q7. Complaint details

Purpose / concept: Three-table join for complaint tracking.

```
SELECT c.complaint_id,
       s.first_name || ' ' || s.last_name AS student_name,
       c.description, c.complaint_date, c.status,
       w.first_name || ' ' || w.last_name AS warden_name
FROM complaint c
JOIN student s ON c.student_id = s.student_id
JOIN warden w ON c.warden_id = w.warden_id;
```

Q8. Unresolved complaints

Purpose / concept: Filters open complaints; status 0 is open in the supplied queries.

```
SELECT complaint_id, student_id,
       description, complaint_date
FROM complaint
WHERE status = 0;
```

Q9. Count students in each room

Purpose / concept: LEFT JOIN keeps rooms even when they have no student.

```
SELECT r.room_no,
       r.capacity,
       COUNT(s.student_id) AS students_count
FROM room r
LEFT JOIN student s
ON r.room_id = s.room_id
GROUP BY r.room_no, r.capacity
ORDER BY r.room_no;
```

Q10. Rooms with available capacity

Purpose / concept: Uses GROUP BY and HAVING to identify free beds.

```
SELECT r.room_no,
       r.capacity,
       COUNT(s.student_id) AS occupied,
       r.capacity - COUNT(s.student_id) AS available
FROM room r
LEFT JOIN student s
ON r.room_id = s.room_id
GROUP BY r.room_no, r.capacity
HAVING COUNT(s.student_id) < r.capacity;
```

Q11. Count students in each course

Purpose / concept: Aggregates hostel strength by course.

```
SELECT course,
       COUNT(*) AS total_students
FROM student
GROUP BY course
ORDER BY total_students DESC;
```

Q12. Total fee collected

Purpose / concept: Calculates collected fees; the supplied output reports 329000.

```
SELECT SUM(amount) AS total_fee_collected
FROM fee_payment
WHERE payment_status = 1;
```

Q13. Visitor log

Purpose / concept: Produces a chronological visitor log.

```
SELECT v.visitor_id, v.visitor_name,
       v.visitor_date,
```

```

        s.student_id,
        s.first_name || ' ' || s.last_name AS student_name
FROM visitor v
JOIN student s
ON v.student_id = s.student_id
ORDER BY v.visitor_date;

```

Q14. Visitors per student

Purpose / concept: Counts visitor records per student.

```

SELECT s.student_id,
       s.first_name || ' ' || s.last_name AS student_name,
       COUNT(v.visitor_id) AS visitor_count
FROM student s
JOIN visitor v
ON s.student_id = v.student_id
GROUP BY s.student_id, s.first_name, s.last_name
ORDER BY visitor_count DESC;

```

Q15. Complaint resolutions

Purpose / concept: Joins complaints to their resolution history.

```

SELECT c.complaint_id, c.description,
       cr.resolution_date, cr.remarks
FROM complaint c
JOIN complaint_resolution cr
ON c.complaint_id = cr.complaint_id
ORDER BY cr.resolution_date;

```

10.1 Key Outputs from the Supplied Data

Query / Measure	Supplied Result
CSE students	7 students
Paid fee records	13 students
Pending fee records	2 students
Total fee collected	329,000
Outstanding dues	52,000 across 2 students
Open complaints	4 complaints
Course distribution	CSE 7, IT 3, AIML 3, ECE 2
Rooms with free beds	10 rooms identified by the supplied HAVING query
Visitors	15 visitor records
Complaint resolutions	15 resolution records

The outputs above are taken from the supplied presentation/code. The exact displayed rows may depend on the state of the Oracle database when the queries are executed.

11. CONCLUSION

The Student Hostel Management System converts common hostel administration activities into a structured relational database. The supplied implementation separates core entities from multi-valued and repeating facts, uses primary and foreign keys for referential integrity, and applies normalization through 1NF, 2NF and 3NF.

The SQL implementation demonstrates practical DBMS concepts including DDL, ALTER TABLE, primary keys, foreign keys, UNIQUE constraints, composite keys, INSERT statements, JOIN operations, GROUP BY, HAVING, COUNT and SUM. The query set provides reports for room allocation, fee status, complaints, course distribution and visitor tracking.

12. FUTURE ENHANCEMENTS

- Role-based access for administrators, wardens, students and security staff.
- Automatic room-allocation procedure or trigger based on available capacity.
- Attendance, leave and hostel-exit management.
- Fee reminders, scholarships and online payment integration.
- Timetable and facility/service management.
- Automated dashboards for occupancy, dues and complaint status.
- Backup, recovery and audit logging.
- Web or mobile interface connected to the Oracle database.

13. REFERENCES

1. Silberschatz, A., Korth, H. F., & Sudarshan, S., *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2020.
2. Elmasri, R., & Navathe, S. B., *Fundamentals of Database Systems*, 7th Edition, Pearson.
3. Oracle Corporation, *Oracle Database SQL Language Reference*, Oracle Database 21c Documentation.
4. Oracle Corporation, *Oracle Database SQL Language Reference*, Oracle Database 19c Documentation.
5. Oracle Corporation, *Oracle Database Documentation*, Oracle Database Technical Documentation.
6. Coronel, C., & Morris, S., *Database Systems: Design, Implementation, & Management*, Cengage Learning.
7. Date, C. J., *An Introduction to Database Systems*, Pearson.

APPENDIX A — STATUS VALUES USED IN SUPPLIED QUERIES

Field	Value	Meaning in supplied project
fee_payment.payment_status	1	Paid
fee_payment.payment_status	0	Pending
complaint.status	1	Resolved
complaint.status	0	Open
room.status	1 / 0	Room status flag; the supplied data uses both values.

APPENDIX B — PROJECT TEAM

- Nunna Sai Sri Laxmi — 25B11CS682
- Kamuju Sree Reshma Laxmi — 25B11CS385
- Bheemireddy Aswini — 25B11CS107
- Medepalli Akshaya Keerthana — 25B11CS592